

PRÀCTICA 4

Mario VILAR
20392724

David Díez
20392643

7 de maig de 2023



UNIVERSITAT DE
BARCELONA

ÍNDEX

1 Introducció	2	4 Conclusions	5
Exercici 1	2	A Programa	5
Exercici 2	2	Codi A.1	6
Observació 3	2	Codi A.2	8
2 Recursos i configuració	3	Codi A.3	13
2.1 Versió emulada	3	Codi A.4	17
2.2 Versió física	3	Codi A.5	18
Observació 4	3	B Diagrames de flux	19
2.3 Configuració	3	Figura B.1	19
2.3.1 Configuració de la UCS	3	Figura B.2	20
2.3.2 Configuració de la UART	4	Figura B.3	21
3 Problemes i solucions	4	Figura B.4	22

INTRODUCCIÓ

L'objectiu és adquirir habilitats per programar un robot utilitzant les funcions bàsiques de comunicació i creant una llibreria de funcions personalitzada per al control del mateix. Durant la pràctica, s'exploraran diferents tècniques i eines per al disseny i programació de robots, incloent l'ús de llenguatges de programació i la integració de sensors i actuadors.

Exercici 1 (Tasca a realitzar).

1. *Generar els recursos bàsics per comunicar la placa del microcontrolador amb el Robot.*
 - Fer una funció per configurar les comunicacions amb els mòduls del Robot.
 - Fer una funció per Enviar «comandaments» als mòduls del Robot.
 - Fer una funció per Rebre informació dels mòduls del Robot.
2. *Fer una llibreria de funcions que permetin controlar el robot.*

I la dividirem en dos parts:

1. *Versió emulada: no ens comunicarem amb els motors/sensor Dynamixel real sinó en una versió emulada en el PC. Per aconseguir-ho ens caldrà usar la UART del microcontrolador que va cap el PC. És recomana començar per aquesta donat que serà molt més senzill detectar errors i corregir-los.*
2. *Versió real: ens comunicarem amb els motors/sensors Dynamixel real. Caldrà canviar a un altre UART que és la que realment està connectada a aquests dispositius. A diferència del cas anterior, cal usar el pin DIRECTION_PORT per alternar entre la recepció i transmissió.*

Exercici 2.

1. *A partir del que s'ha estudiat a la classe de teoria, i dels Manuals del AX-12 o el AX-SI, analitzem les funcions de transmissió de dades (TXD) i estudiem el seu funcionament. Comentem les instruccions que no ho estan i expliquem tot això a l'informe de la pràctica.*
2. *Fem una funció similar per tal de rebre dades (RXD) dels mòduls Dynamixel, utilitzant en aquest cas la capacitat de la USCI de generar interrupcions en recepció.*

Observació 3. En el emulador no tenim disponible els dos modes de moviment del motor (moviment angular o moviment continu), sinó que només el mode continu. En aquest mode, referit com a endless turn en el manual, definim una velocitat i un sentit de gir pel motor i aquest el continuarà aplicant contínuament fins que s'indiqui un altre valor. Aquest és l'únic mode disponible pel emulador i el que ja ve configurat per defecte. És també el mode de funcionament recomanat per la pràctica i el projecte. 

RECURSOS I CONFIGURACIÓ

Com ja sabem, la comunicació entre el microcontrolador i els mòduls del robot (*AX-12: Motors; AX-SI: Sensor*) és fa mitjançant un protocol sèrie asíncron. El recurs del processador que s'encarrega d'això es la USCI (*Universal Serial Communication Interface*) programada com a UART (*Universal Asynchronous Receiver Transmitter*). El nostre microcontrolador disposa de 4 USCIs que poden funcionar com a UART.

2.1

VERSIÓ EMULADA

En el cas de la versió emulada els motors tindran IDs 1 i 2 i el sensor ID 3. L'emulació d'aquests s'executarà en una aplicació python que emularà el seu funcionament. Aquesta aplicació s'executarà en el ordinador on connectem la placa d'avaluació i és comunicarà pel USB amb el microcontrolador MSP432. Per part del microcontrolador, la comunicació és realitzarà amb el debugger integrat en la placa d'avaluació, i aquest farà la translació entre UART i USB. En resum, per la part del codi que hem de desenvolupar, el protocol de comunicació serà UART.

2.2

VERSIÓ FÍSICA

La USCI a usar és la UCA2, que està al port 3 (pins P3.2 = RX, P3.3 = TX). En el cas de la versió real els motors tindran IDs 1 i 2 i el sensor ID 100. Els identificadors poden ser uns altres depenent del robot. En tot cas disposen d'etiquetes que, excepte que s'hagin accidentalment reprogramat, mostren quin hauria de ser el ID correcte del dispositiu.

Observació 4. En cap cas és pretén tenir dos codis diferents. L'objectiu és basa en fer ús de defines i/o variables per tal de poder seleccionar en el codi amb quina de les dos versions és vol treballar. ↻

2.3

CONFIGURACIÓ

La configuració general serà la següent:

1. Configuració del sistema de rellotge del microcontrolador UCS (Unified Clock System) per que la UART treballi a la freqüència requerida pels mòduls Dynamixel.
2. Configuració de la UART.
3. Funcions de transmissió i recepció d'una trama tal com la requereixen els mòduls Dynamixel.

2.3.1 Configuració de la UCS

La UCS configura 3 fonts de rellotge:

1. **MCLK:** Master Clock per a la CPU.

2. **SMCLK**: Secondary Master Clock disponible pels perifèrics.
3. **ACLK**: Auxiliary Clock disponible pels perifèrics.

Pensem que la configuració de la UCS repercuteix en el funcionament de tot el sistema, per tant es recomana que aquesta configuració es faci al començament de l'execució del programa (just després d'aturar el “Watch-dog Timer” tal com hem anat fent en les pràctiques anteriors)

2.3.2 Configuració de la UART

La configuració de la UART hauria d'ajustar-se al sistema de comunicació dels mòduls Dynamixel. El protocol que fan servir aquests mòduls és sèrie, asíncron de 8 bits, 1 bit de stop i sense paritat. A més, per nosaltres, les comunicacions amb l'emulador de la pràctica funcionen amb un baud rate de 500 kbps. No obstant, per millorar la detecció dels bits en recepció, farem treballar la nostra UART amb sobremostreig (oversampling), agafant 16 mostres per cada bit transmès. Això vol dir que una UART que vulgui treballar a aquest baud rate ha d'estar alimentada amb un rellotge 16 vegades més ràpid.

Tenint en compte tots aquest requeriments, estudiem la configuració de la UART a partir dels valors assignats als registres de configuració adjacents en la funció `init_uart()` proporcionada a classe de teoria, i expliquem-lo a l'informe (recordem que com a part de les eines de depuració, el CCS disposa d'una pestanya `Registers`, que ens permet inspeccionar el contingut dels registres del microcontrolador). Les funcions proporcionades a classe de teoria serveixen per transmetre una trama d'instruccions amb el format dels mòduls Dynamixel.

Hem de tenir en compte que ara des del microcontrolador, per programa, hem de controlar el senyal `DIRECTION_PORT`. A la nostra placa l'hem connectat al pin `GPIO P3.0`. Hem d'inicialitzar aquest senyal i gestionar-ho en funció de si volem enviar o rebre missatges a les funcions corresponents de transmetre o rebre. Per això farem un parell de funcions que controlin aquest pin i les farem servir just quan comencem una transmissió o una recepció. Finalment, aquesta línia “Data”, l'hauré de mantenir sempre per defecte en l'estat `RXD`. La canviarem a `TXD` només quan vulguem transmetre-li dades a algun mòdul, i la tornarem a `RXD` quan s'hagi acabat de transmetre l'últim bit de la trama a enviar.

3

PROBLEMES I SOLUCIONS

Els nostres problemes, sobretot, s'han donat a l'hora de:

1. canviar entre emulador i robot, ja que sempre ens deixàvem de canviar algun senyal de port;
2. entendre totes les interrupcions que havíem d'emetre, la inicialització de diversos senyals i components que no coneixíem (EUSCI...), que hem solucionat comprovant el manual;
3. el traspàs del codi a llibreries, ja que teníem un `main` 100% funcional, però concentrat en 500 línies... hi ha certs components que ens han deixat d'anar, i com a solució, el fet de prendre-s'ho amb molta

paciència i anar revisant línia per línia que no faltés cap `include`, que totes les funcions quedessin declarades al `header`, així com haver declarat totes les variables internes en el mateix arxiu que acabem utilitzant-les.

4

CONCLUSIONS

Durant la pràctica, hem après a programar un robot utilitzant diferents funcions de comunicació i han creat una llibreria de funcions personalitzada per al control del mateix. A través d'aquesta pràctica, s'han explorat les diferents tècniques i eines necessàries per al disseny i programació de robots, hem adquirit habilitats per programar i controlar un robot de manera autònoma, aplicant els conceptes i habilitats adquirides en l'assignatura. A més, hem aprofundit en els nostres coneixements de diagrames de fluxos, ja que cada vegada els nostres programes van adquirint més components i matisos.

En resum, la pràctica ha estat una experiència pràctica i enriquidora per a nosaltres, que ens ha permès comprendre millor el funcionament dels robots i ens ha proporcionat una base sòlida per al nostre futur i últim projecte en el camp de la robòtica: el treball final.

A

PROGRAMA

No entrarem en detalls en l'explicació del programa perquè ens hem assegurat de comentar-lo al més acuradament possible. Dividirem, això sí, els codis en les diferents llibreries que hem creat. **Tampoc enganxarem els fitxers de capçalera, `.h`, ja que simplement declaren les variables i funcions que utilitzarem en el seu respectiu `.c`. Es podrà consultar tot a l'entrega de la pràctica.**

```
1  #include <msp432p401r.h>
2  #include <stdio.h>
3  #include <stdint.h>
4  #include <stdbool.h>
5
6  // Cridem a totes les llibreries que hem creat. Cadascuna fa una funció diferent
7  // Moviments guarda els moviments que pot fer el robot
8  // Rxtx guarda la transferència de dades i la configuració de la UART
9  // Timer guarda la inicialització del timer
10 // Interrupcions guarda la gestió de les interrupcions
11 #include "lib_PAE.h"
12 #include "moviments.h"
13 #include "Rxtx.h"
14 #include "timer.h"
15 #include "interrupcions.h"
```

```
16
17
18 // Cridem el main.
19 void main(void)
20 {
21     WDT_A->CTL = WDT_A_CTL_PW | WDT_A_CTL_HOLD;
22
23     //Inicialitzem les dades com fa falta
24     init_ucs_24MHz();
25
26     // Configurem els Timers
27     init_timers();
28
29     // Configurem la UART
30     init_UART();
31
32     // Configurem les interrupcions
33     init_interrupciones();
34     __enable_interrupts();
35
36     // Funcions per encendre els LEDS del motor
37     left_led();
38     right_led();
39
40     // Definim la velocitat a la que es moura el robot
41     int speed = 0x100;
42
43     // Funció per fer que el robot es pugui moure
44     set_up_velocity();
45
46
47     // Definim els moviments que volem que el robot faci
48     endavant(speed);
49     while (1);
50 }
```

Codi A.1: Codi del nostre *main*.

```
1 #include <msp432p401r.h>
2 #include <stdio.h>
3 #include <stdint.h>
4 #include <stdbool.h>
5
6 #include "lib_PAE.h"
7 #include "moviments.h"
8 #include "Rxtx.h"
```

```
9  #include "timer.h"
10 #include "interrupcions.h"
11
12 // Inicialitzem les interrupcions pel robot
13 void init_interr_robot(void){
14     NVIC->ICPR[0] |= 10000000000;
15     NVIC->ISER[0] |= 10000000000;
16
17     NVIC->ICPR[0] |= 1 << EUSCIA0_IRQn;
18     NVIC->ISER[0] |= 1 << EUSCIA0_IRQn;
19
20     NVIC->ICPR[0] |= 1 << EUSCIA2_IRQn;
21     NVIC->ISER[0] |= 1 << EUSCIA2_IRQn;
22
23
24     NVIC->ICPR[1] |= 1 << (PORT3_IRQn & 31);
25     NVIC->ISER[1] |= 1 << (PORT3_IRQn & 31);
26 }
27
28 // Inicialitzem les interrupcions per l'emulador
29 void init_interr_emulador(void){
30     NVIC->ICPR[0] |= 1 << TA1_0_IRQn;
31     NVIC->ISER[0] |= 1 << TA1_0_IRQn;
32
33     NVIC->ICPR[0] |= 1 << EUSCIA0_IRQn;
34     NVIC->ISER[0] |= 1 << EUSCIA0_IRQn;
35
36     NVIC->ICPR[1] |= 1 << (PORT3_IRQn & 31);
37     NVIC->ISER[1] |= 1 << (PORT3_IRQn & 31);
38 }
39
40
41 // Configurem les interrupcions
42 void init_interrupciones() {
43     init_interr_robot();
44     //init_interr_emulador();
45 }
46
47
48 // Recepció de bytes utilitzant interrupcions
49 void EUSCIA0_IRQHandler(void)
50 {
51     EUSCIA0->IFG &= ~ EUSCIA0_IFG_RXIFG;
52     UCA0IE &= ~UCRXIE;
53     DatoLeido_UART = UCA0RXBUF;
54     Byte_Recibido=1;
55     UCA0IE |= UCRXIE;
```

```

56 }
57
58 // Recepció de bytes utilitzant interrupcions
59 void EUSCIA2_IRQHandler(void)
60 {
61     EUSCI_A2->IFG &=~ EUSCI_A_IFG_RXIFG;
62     UCA2IE &= ~UCRXIE;
63     DatoLeido_UART = UCA2RXBUF;
64     Byte_Recibido=1;
65     UCA2IE |= UCRXIE;
66 }

```

Codi A.2: Codi de les nostres *Interrupcions*.

```

1  #include <msp432p401r.h>
2  #include <stdio.h>
3  #include <stdint.h>
4  #include <stdbool.h>
5
6  // Cridem a totes les llibreries que hem creat. Cadascuna fa una funció diferent
7  #include "lib_PAE.h"
8  #include "moviments.h"
9  #include "RxTx.h"
10 #include "timer.h"
11 #include "interrupcions.h"
12
13 // Set_up_velocity s'encarrega de fer que el moviment de les rodes sigui constant
14 void set_up_velocity(void){
15     // Definim els valors que hi haurà a l'array RxParam, és el que li enviarem des del robot
16     byte RxParam[16];
17     RxParam[0] = CW_ANGLE_LIMIT_L;
18     RxParam[1] = 0x00;
19     RxParam[2] = 0x00;
20     RxParam[3] = 0x00;
21     RxParam[4] = 0x00;
22
23     RxReturn respuesta;
24     byte bPacket;
25
26     // Mentre el checksum sigui correcte llegim les dades
27     respuesta.checkChecksum = 1;
28     while(respuesta.checkChecksum){
29         // Enviem la instrucció
30         bPacket = TxPacket(0x02, 0x05, 0x03, RxParam);
31         respuesta = RxPacket();
32     }

```

```
33     respuesta.checkChecksum = 1;
34     while(respuesta.checkChecksum){
35         // Enviem la instrucció
36         bPacket = TxPacket(0x03, 0x05, 0x03, RxParam);
37         respuesta = RxPacket();
38     }
39
40 }
41
42 // Funció per fer que el robot es mogui cap endavant
43 void endavant(int speed){
44     RxReturn respuesta;
45     byte bPacket;
46
47     // La direcció del motor2 ser 1
48     byte direction = 1;
49
50     // Definim els valors de RxParam
51     byte RxParam1[16];
52     RxParam1[0] = 0x20;
53
54     // Utilitzem el valor speed desitjat
55     RxParam1[1] = speed & 0xFF;
56     RxParam1[2] = ((direction<<2)) | ((speed>>8)&0x03);
57
58     bPacket = TxPacket(0x02, 0x03, 0x03, RxParam1);
59
60     // Mentre el checksum sigui correcte llegim les dades
61     respuesta.checkChecksum = 1;
62     while(respuesta.checkChecksum){
63         // Enviem la instrucció
64         bPacket = TxPacket(0x02, 0x03, 0x03, RxParam1);
65         respuesta = RxPacket();
66     }
67
68     // La direcció del motor3 ser 0
69     direction = 0;
70     RxParam1[2] = ((direction<<2)&4) | ((speed>>8)&0x03);
71
72     // Mentre el checksum sigui correcte llegim les dades
73     respuesta.checkChecksum = 1;
74     while(respuesta.checkChecksum){
75         // Enviem la instrucció
76         bPacket = TxPacket(0x03, 0x03, 0x03, RxParam1);
77         respuesta = RxPacket();
78     }
79 }
```

```
80
81 // Funci per fer que el robot es mogui cap enrere
82 void enrere(int speed){
83     RxReturn respuesta;
84     byte bPacket;
85
86     // La direcció del motor2 ser 0
87     byte direction = 0;
88
89     // Definim els valor de RxParam
90     byte RxParam1[16];
91     RxParam1[0] = 0x20;
92
93     // Utilitzem el valor speed desitjat
94     RxParam1[1] = speed & 0xFF;
95     RxParam1[2] = ((direction<<2)) | ((speed>>8)&0x03);
96
97     bPacket = TxPacket(0x02, 0x03, 0x03, RxParam1);
98
99     // Mentre el checksum sigui correcte llegim les dades
100    respuesta.checkChecksum = 1;
101    while(respuesta.checkChecksum){
102        // Enviem la instrucció
103        bPacket = TxPacket(0x02, 0x03, 0x03, RxParam1);
104        respuesta = RxPacket();
105    }
106
107    // La direcció del motor3 ser 1
108    direction = 1;
109    RxParam1[2] = ((direction<<2)&4) | ((speed>>8)&0x03);
110
111    // Mentre el checksum sigui correcte llegim les dades
112    respuesta.checkChecksum = 1;
113    while(respuesta.checkChecksum){
114        // Enviem la instrucció
115        bPacket = TxPacket(0x03, 0x03, 0x03, RxParam1);
116        respuesta = RxPacket();
117    }
118 }
119
120 // Funci per fer que giri a l'esquerra. Hi ha dos velocitats, una per a cada roda.
121 // ↩ D'aquesta forma pot no girar simètricament
122 void gir_esquerra(int speed1, int speed2){
123     RxReturn respuesta;
124     byte bPacket;
125
126     // La direcció del motor2 ser 0
```

```
126     byte direction = 0;
127
128     // Definim els valor de RxParam
129     byte RxParam1[16];
130     RxParam1[0] = 0x20;
131
132     // La roda d'aquest motor girar a la velocitat speed1
133     RxParam1[1] = speed1 & 0xFF;
134     RxParam1[2] = ((direction<<2)) | ((speed1>>8)&0x03);
135
136     bPacket = TxPacket(0x02, 0x03, 0x03, RxParam1);
137
138     // Executem mentre el checksum sigui correcte
139     respuesta.checkChecksum = 1;
140     while(respuesta.checkChecksum){
141         // Enviem la instrucció
142         bPacket = TxPacket(0x02, 0x03, 0x03, RxParam1);
143         respuesta = RxPacket();
144     }
145
146     // La direcció del motor3 ser 0
147     direction = 0;
148
149     // La roda d'aquest motor girar a la velocitat speed2
150     RxParam1[1] = speed2 & 0xFF;
151     RxParam1[2] = ((direction<<2)&4) | ((speed2>>8)&0x03);
152
153     // Executem mentre el checksum sigui correcte
154     respuesta.checkChecksum = 1;
155     while(respuesta.checkChecksum){
156         // Enviem la instrucció
157         bPacket = TxPacket(0x03, 0x03, 0x03, RxParam1);
158         respuesta = RxPacket();
159     }
160 }
161
162 // Funció per fer que giri a la dreta. Hi ha dos velocitats, una per a cada roda. D'aquesta
163 ↪ forma pot no girar simètricament
164 void gir_dreta(int speed1, int speed2){
165     RxReturn respuesta;
166     byte bPacket;
167
168     // La direcció del motor2 ser 1
169     byte direction = 1;
170
171     // Definim els valor de RxParam
172     byte RxParam1[16];
```

```
172 RxParam1[0] = 0x20;
173
174 // La roda d'aquest motor girar a la velocitat speed1
175 RxParam1[1] = speed1 & 0xFF;
176 RxParam1[2] = ((direction<<2)) | ((speed1>>8)&0x03);
177
178 bPacket = TxPacket(0x02, 0x03, 0x03, RxParam1);
179
180 // Executem el bucle mentre el check sum sigui correcte
181 respuesta.checkChecksum = 1;
182 while(respuesta.checkChecksum){
183     // Enviem la instrucció
184     bPacket = TxPacket(0x02, 0x03, 0x03, RxParam1);
185     respuesta = RxPacket();
186 }
187
188 // La direcció del motor3 ser 1
189 direction = 1;
190
191 // La roda d'aquest motor girar a la velocitat speed2
192 RxParam1[1] = speed2 & 0xFF;
193 RxParam1[2] = ((direction<<2)&4) | ((speed2>>8)&0x03);
194
195 // Executem el bucle mentre el check sum sigui correcte
196 respuesta.checkChecksum = 1;
197 while(respuesta.checkChecksum){
198     // Enviem la instrucció
199     bPacket = TxPacket(0x03, 0x03, 0x03, RxParam1);
200     respuesta = RxPacket();
201 }
202 }
203
204 // Funció per encendre el LED del motor esquerre
205 void left_led(void) {
206     // Configurem la instrucció
207     byte bID = 0x02;
208     byte bParameterLength = 0x02;
209     byte bInstruction = 0x03;
210     byte Parametros[16];
211     Parametros[0] = 0x19;
212     Parametros[1] = 0x01;
213
214     // Envem la instrucció
215     TxPacket(bID, bParameterLength, bInstruction, Parametros);
216
217     RxReturn ret = RxPacket();
218     UCA0IE |= UCRXIE;
```

```
219     Sentit_Dades_Rx();
220 }
221
222 // Funci per encendre el LED del motor dret
223 void right_led(void) {
224     // Configurem la instrucció
225     byte bID = 0x03;
226     byte bParameterLength = 0x02;
227     byte bInstruction = 0x03;
228     byte Parametros[16];
229     Parametros[0] = 0x19;
230     Parametros[1] = 0x01;
231
232     // Envem la instrucció
233     TxPacket(bID, bParameterLength, bInstruction, Parametros);
234
235     RxReturn ret = RxPacket();
236     UCA0IE |= UCRXIE;
237     Sentit_Dades_Rx();
238 }
```

Codi A.3: Codi cabdal, segurament, de la nostra pràctica: *Moviments*.

```
1  #include <msp432p401r.h>
2  #include <stdio.h>
3  #include <stdint.h>
4  #include <stdbool.h>
5
6  #include "lib_PAE.h"
7  #include "moviments.h"
8  #include "RxTx.h"
9  #include "timer.h"
10 #include "interrupcions.h"
11
12 // Configurem la UART per l'emulador. Utilitzem ID0
13 void init_UART_emulador(void){
14     UCA0CTLW0 |= UCSWRST;
15     UCA0CTLW0 |= UCSSEL__SMCLK;
16
17     UCA0MCTLW = UCOS16;
18     UCA0BRW = 13;
19     UCA0MCTLW |= (0x25 << 8);
20
21
22     // Configurem els pins de la UART.
23     // El port P3.2 correspon a UART2RX i el port P3.3 correspon a UART2TX
```

```

24 P1SELO |= (BIT2 | BIT3);
25 P1SEL1 &= ~(BIT2 | BIT3);
26 UCAOCTLW0 &= ~UCSWRST;
27
28 EUSCI_A0->IFG &= ~EUSCI_A_IFG_RXIFG;
29 EUSCI_A0->IE |= EUSCI_A_IE_RXIE;
30
31 //El pin 0 del port 3 indica la direcció de data 1 per TX i 0 per RX
32 P3SELO &= ~BIT0;
33 P3SEL1 &= ~BIT0;
34 P3DIR |= BIT0;
35
36 // Recepció de dades
37 Sentit_Dades_Rx();
38 }
39
40
41 // Configurem la UART per l'emulador. Utilitzem ID2
42 void init_UART(void){
43     UCA2CTLW0 |= UCSWRST;
44     UCA2CTLW0 |= UCSSEL__SMCLK;
45
46     UCA2MCTLW = UCOS16;
47     UCA2BRW = 3;
48
49     // Configurem els pins de la UART.
50     // El port P3.2 correspon a UART2RX i el port P3.3 correspon a UART2TX
51     P3SELO |= (BIT2 | BIT3);
52     P3SEL1 &= ~(BIT2 | BIT3);
53     UCA2CTLW0 &= ~UCSWRST;
54
55     EUSCI_A2->IFG &= ~EUSCI_A_IFG_RXIFG;
56     EUSCI_A2->IE |= EUSCI_A_IE_RXIE;
57
58
59     //El pin 0 del port 3 indica la direcció de data 1 per TX i 0 per RX
60     P3SELO &= ~BIT0;
61     P3SEL1 &= ~BIT0;
62     P3DIR |= BIT0;
63
64     // Recepció de dades
65     Sentit_Dades_Rx();
66 }
67
68 // Configurem el packet que envia el robot
69 byte TxPacket(byte bID, byte bParameterLength, byte bInstruction, byte Parametros[16])
70 {

```

```

71     byte bCount,bChecksum,bPacketLength;
72     byte TxBuffer[32];
73
74     Sentit_Dades_Tx();
75
76     TxBuffer[0] = 0xff;
77     TxBuffer[1] = 0xff;
78     TxBuffer[2] = bID;
79     TxBuffer[3] = bParameterLength+2;
80     TxBuffer[4] = bInstruction;
81
82     char error[] = "adr. no permitida";
83
84     // Si es vol escriure una direcció <= 0x05 obtenim un error
85     if ((Parametros[0] < 6) && (bInstruction == 3)){
86         hallCdPrintLine(error, 8, INVERT_TEXT);
87         return 0;
88     }
89
90     for(bCount = 0; bCount < bParameterLength; bCount++) {
91         TxBuffer[bCount+5] = Parametros[bCount];
92     }
93
94     bChecksum = 0;
95     bPacketLength = bParameterLength+4+2;
96
97     for(bCount = 2; bCount < bPacketLength-1; bCount++) {
98         bChecksum += TxBuffer[bCount];
99     }
100
101     TxBuffer[bCount] = ~bChecksum;
102
103     for (bCount = 0; bCount < bPacketLength; bCount++) //Aquest bucle s'envia la
↪ trama al Mdul Robot
104     {
105         TxUACx(TxBuffer[bCount]);
106     }
107
108     while( (UCA2STATW & UCBSY) ); // waiting
109     Sentit_Dades_Rx();
110
111     return(bPacketLength);
112 }
113
114 // Configurem el packet que rep el motor
115 RxReturn RxPacket(void)
116 {

```

```
117 RxReturn respuesta;
118 byte bCount, bLength, bChecksum;
119 byte Rx_time_out=0;
120 respuesta.TimeOut = Rx_time_out;
121
122 UCA2IE |= UCRXIE;
123
124 Sentit_Dades_Rx();
125
126 for(bCount = 0; bCount < 4; bCount++)
127 {
128     Reset_Timeout();
129     Byte_Recibido=0;
130
131     while (!Byte_Recibido)
132     {
133         Rx_time_out=Timeout(1000);
134
135         if (Rx_time_out) break;
136     }
137
138     if (Rx_time_out){
139         respuesta.TimeOut = 1;
140         break;
141     }
142
143     respuesta.StatusPacket[bCount] = DatoLeido_UART;
144 }
145
146
147 // Continua llegint la resta de bytes del Status Packet si no ha saltat el Timeout
148 if (!Rx_time_out)
149 {
150     bLength = respuesta.StatusPacket[3] + 4;
151
152     for(bCount = 4; bCount < bLength; bCount++){
153         Reset_Timeout();
154         Byte_Recibido=0;
155
156         while(!Byte_Recibido){
157             Rx_time_out=Timeout(1000);
158
159             if(Rx_time_out) break;
160         }
161
162         if(Rx_time_out){
163             respuesta.TimeOut = 1;
```

```
164         break;
165     }
166
167     respuesta.StatusPacket[bCount] = DatoLeido_UART;
168 }
169 }
170
171 // Comprovem que les dades s'hagin llegit correctament
172 bChecksum = 0;
173
174 if(!Rx_time_out){
175
176     for(bCount = 2; bCount < bLength - 1; bCount++){
177         bChecksum += respuesta.StatusPacket[bCount];
178     }
179     bChecksum = ~bChecksum;
180     if (bChecksum != respuesta.StatusPacket[bLength - 1])
181     {
182         respuesta.checkChecksum = 1;
183     } else {
184         respuesta.checkChecksum = 0;
185     }
186 }
187
188 return respuesta;
189 }
190
191 void TxUACx(byte bTxdData)
192 {
193     while (!(UCA2IFG & UCTXIFG)); // Espera a que estigui preparat el buffer de transmissi
194     UCA2TXBUF = bTxdData; //escrivim el byte a ser enviat
195 }
196 }
197
198
199 // Funcions per canviar el sentit de les comunicacions
200 void Sentit_Dades_Rx(void){
201     P3OUT &= ~BIT0;
202 }
203
204 void Sentit_Dades_Tx(void) {
205     P3OUT |= BIT0;
206 }
```

Codi A.4: Codi respecte de l'*RxTx*.

```
1  #include <msp432p401r.h>
2  #include <stdio.h>
3  #include <stdint.h>
4  #include <stdbool.h>
5
6  #include "lib_PAE.h"
7  #include "moviments.h"
8  #include "RtTx.h"
9  #include "timer.h"
10 #include "interrupcions.h"
11
12 // Funcio per inicialitzar els timers
13 void init_timers(void) {
14     TIMER_A1->CTL = TIMER_A_CTL_ID__1 | TIMER_A_CTL_SSEL__ACLK | TIMER_A_CTL_CLR
15         | TIMER_A_CTL_MC__UP;
16     TIMER_A1->CCR[0] = 2399;
17
18     TIMER_A1->CCTL[0] |= TIMER_A_CCTLN_CCIE;
19 }
20
21 // Reset del timeout
22 void Reset_Timeout(void) {
23     counter = 0;
24 }
25
26 // Veiem que hem exedit el timeout
27 int TimeOut(int t) {
28     // Mirem si hem exedit el temps limit
29     if(counter > t){
30         return 1;
31     }
32     return 0;
33 }
34
35 // Interrupci que controla la variable timer segons el clock definit
36 void TA1_0_IRQHandler(void) {
37     TA1CCTL0 &= ~TIMER_A_CCTLN_CCIFG;
38
39     // Augmentem el counter
40     counter++;
41 }
```

Codi A.5: Codi respecte del *timer*.

B

DIAGRAMES DE FLUX

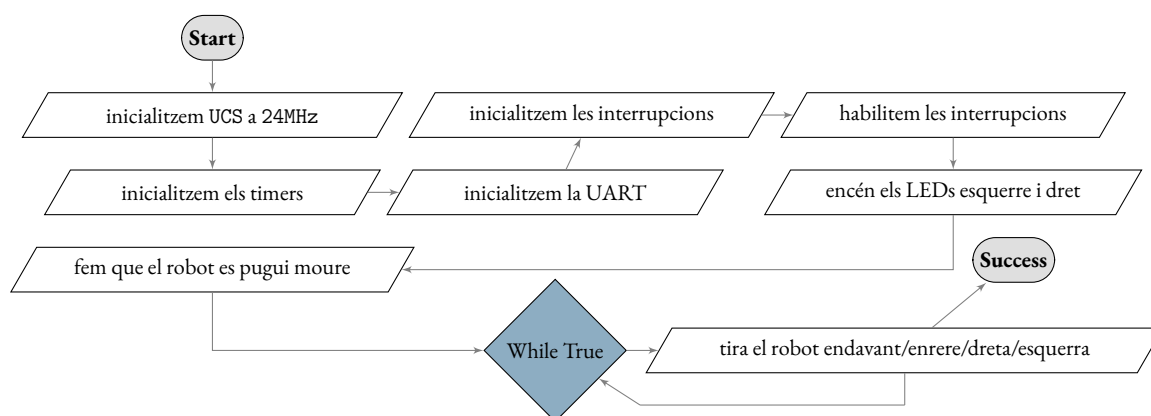


Figura B.1: Diagrama de flux de la funció *main* i del nostre programa en general.

Si ara només estudiem el Diagrama de flux del *main* obtenim B.1, on també tenim una petita idea de com funciona el nostre programa a grans trets. On inicialitzem UCS mitjançant la funció `init_ucs_24MHz()`; d'igual forma, `init_interrupciones()` inicialitza les interrupcions, `init_UART()` inicialitza la UART i `init_timers()` inicialitza els *timers*. Les funcions `left_led()` i `right_led()` configuren els LEDs i `_enable_interrupts()` habilita les interrupcions. Es passa a la part de control del moviment del robot amb un bucle `while`. En aquesta part, entra en moviment el robot en la direcció seleccionada, com ara endavant, enrere, dreta i esquerra¹. El diagrama també mostra que si es mou el robot amb èxit, s'arriba al final del programa i es para l'execució (Success).

¹ En el cas del codi que hem enganxat, A.1, línia 48, veiem que és *endavant*, però es pot intercanviar pel nom que vulguem, per a la direcció que volem que prengui.

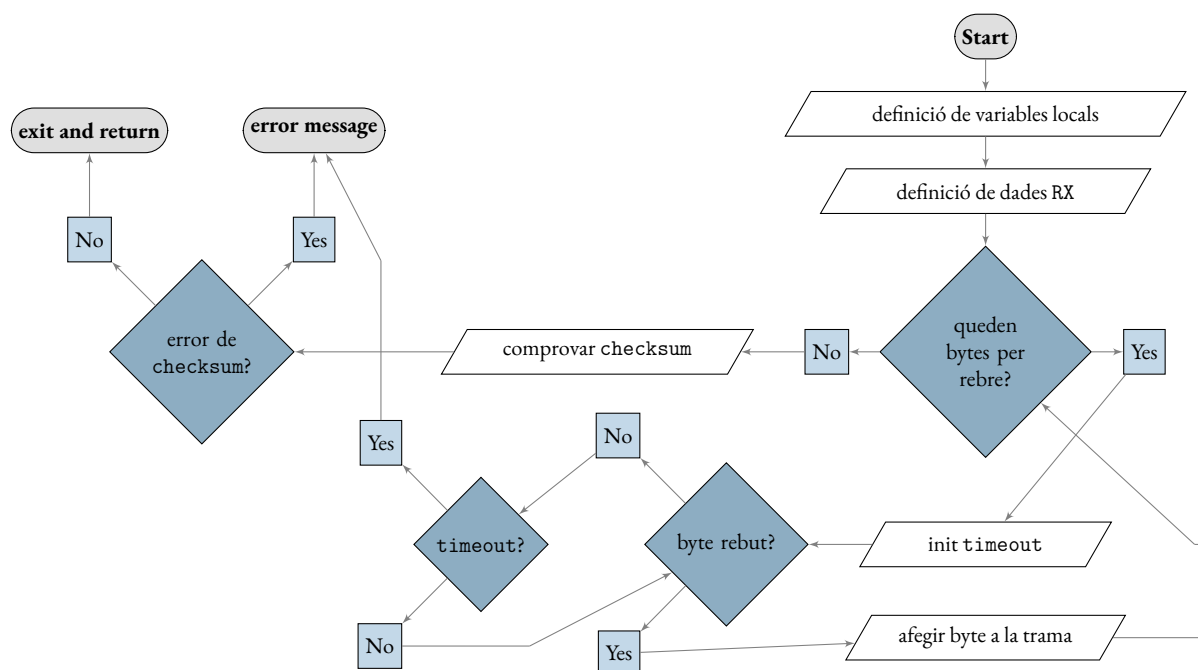


Figura B.2: Funció *Rx* d'una trama *Status* dels mòduls del Robot.

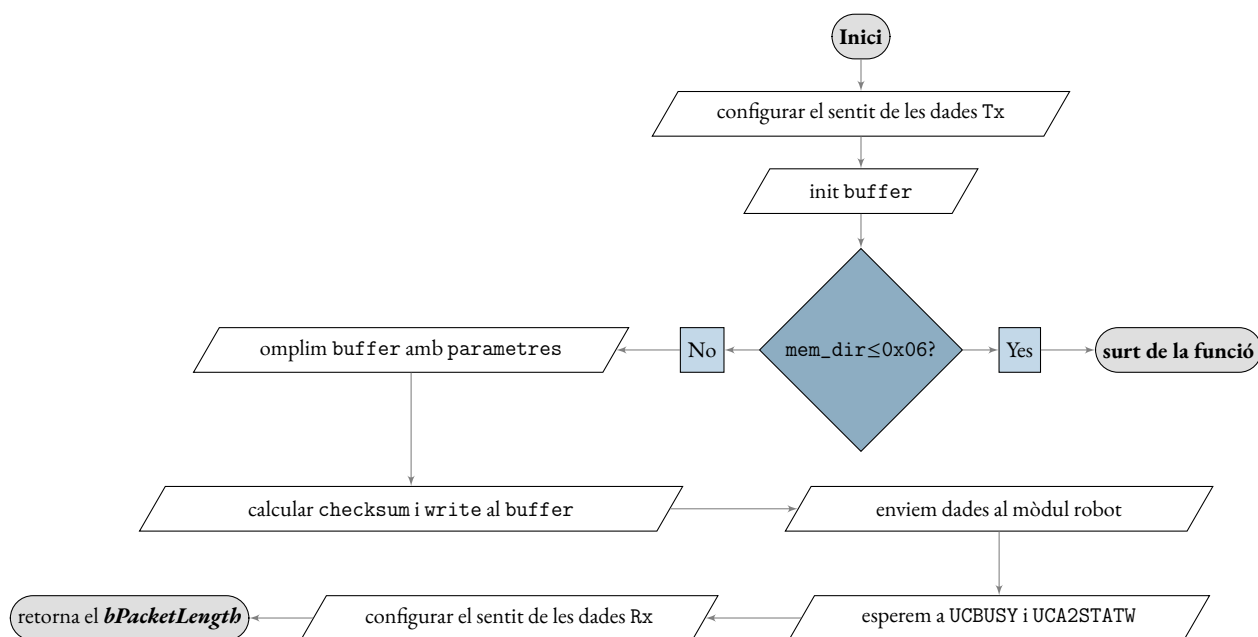
La part superior del diagrama defineix les variables i dades que es necessiten per executar el procés, mentre que la part inferior s'ocupa de la recepció i el processament de les dades.

El procés comença amb la definició de les variables locals i la definició de les dades RX. A continuació, el diagrama inclou una decisió per verificar si queden bytes per rebre. Si hi ha bytes per rebre, es procedeix amb la recepció de dades, si no, es comprova el checksum.

La part central del diagrama es compon d'un bucle que rep les dades i les afegeix a la trama. Si s'ha rebut un byte, es comprova si s'ha produït un timeout. Si no hi ha timeout, es continua rebent dades fins que s'han rebut tots els bytes de la trama. Si hi ha timeout, se surt del bucle i es mostra un missatge d'error.

Finalment, si es produeix un error de checksum o timeout, el diagrama inclou processos per gestionar els errors i sortir del programa.

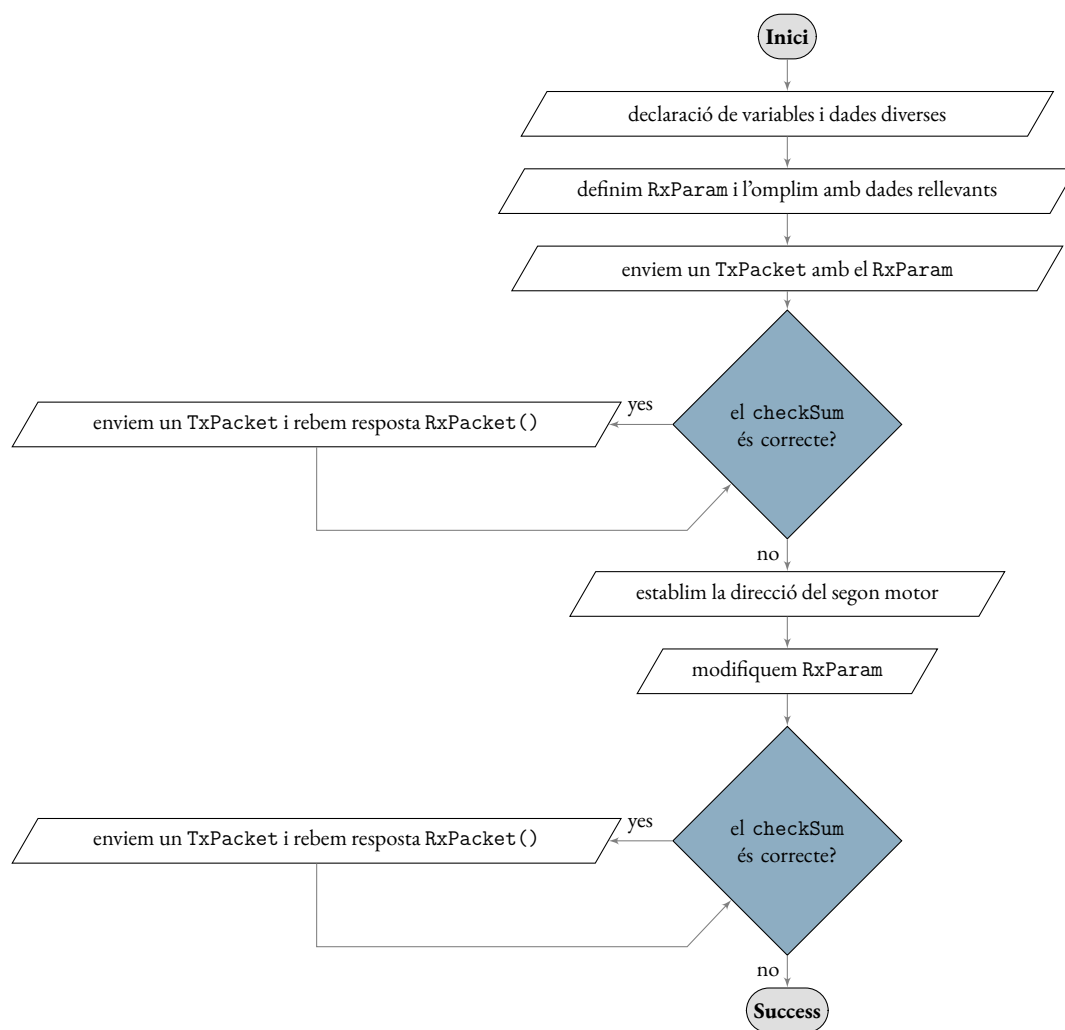
En resum, aquest diagrama proporciona una visió general del procés de recepció de dades i dels diferents camins que el programa pot seguir en funció dels resultats obtinguts.

Figura B.3: Diagrama de flux de la funció *TxPacket*

Tenim un diagrama de flux de dades que mostra el procés per enviar dades a un mòdul robot utilitzant un microcontrolador MSP432. El procés comença amb la configuració del sentit de les dades Tx, després es fa una inicialització del buffer per a l'enviament de dades. A continuació, es comprova si la memòria es troba en una posició menor o igual a 0x06, i si és així, el programa surt de la funció. En canvi, si la memòria es troba en una posició superior a 0x06, el buffer es buida amb els paràmetres per a l'enviament de dades. A continuació, es calcula el checksum i es realitza l'escriptura del buffer. Les dades es envien al mòdul robot, i es espera a que el bus estigui disponible abans de configurar el sentit de les dades Rx. Finalment, la funció retorna el `bPacketLength`.

Per no omplir el document de diagrames, hem optat per desfer-nos d'alguns similars. És el cas de, per exemple, *TxPacket*², *left-led*, *right-led* i totes les direccions que no siguin *endavant*, així com el *set-velocity*, ja que amb veure com hi ha una transmissió Rx-Tx hi ha més que suficient. A part, intentarem ser una mica més breus a partir d'ara per no estendre'ns molt més.

² Acabem de dibuixar el *RxPacket*. Tot i que, en efecte, les diferències són evidents, també comparteixen certs aspectes ja que al final l'un i l'altre són dues cares de la mateixa moneda, aquesta moneda essent la correcta transmissió de les dades en el nostre microcontrolador MSP432.

Figura B.4: Diagrama de flux de la funció *endavant*.

Aquest diagrama de flux descriu el procés d'enviament d'un paquet de dades, TxPacket, a través d'un sistema. Com ja hem dit abans, tots es fan de manera similar, així que ja no en comentarem cap més. Primer, s'inicialitzen les variables i les dades necessàries; entre elles, la direcció del primer motor. A continuació, es defineix i es completa l'estructura RxParam amb les dades rellevants, que es fa servir per a enviar el paquet TxPacket. A partir d'aquí, es comprova si la suma de verificació (checksum) del paquet RxPacket és correcta. Si ho és, s'envia un altre paquet TxPacket amb els mateixos arguments. Si no ho és, significa que ja hem acabat la transmissió, i se surt del bucle per establir la direcció del segon motor. En qualsevol cas, es modifica l'estructura RxParam abans de verificar de nou el checksum, ara per al segon motor. Si és correcta, s'envia un altre paquet TxPacket per confirmar les dades. Si no ho és, el sistema passa a la fase de finalització. En cas d'èxit, el programa acaba amb èxit.